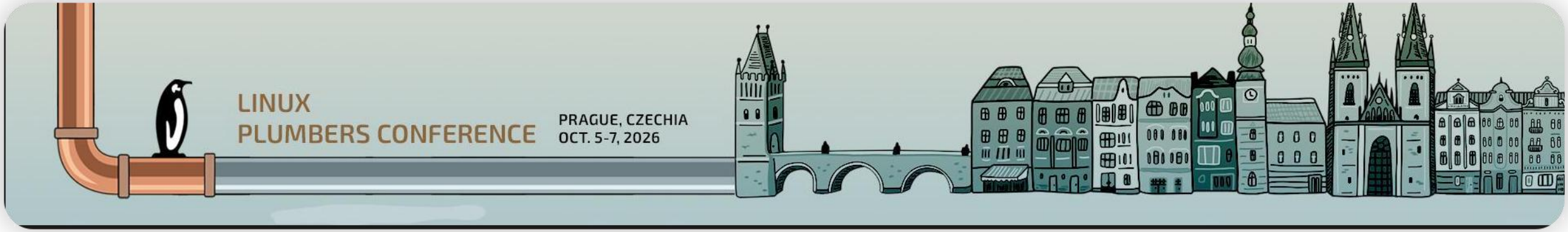
16KB mTHP and large folio pagecache on Android

Nanzhe Zhao/Bo Zhang/Barry Song

Xiaomi



16KB large folios vs. 16KB base page

Pros:

Backward Compatible: Fully supports 4 KB applications

Less Memory Waste: Retains 4 KB as the minimum allocation granularity

Upstream Friendly: Aligned with the Linux community's direction

Cons:

Allocation Risk: Contiguous 16KB allocation may fail

Fragmentation: Prone to memory fragmentation

TLB Overhead: Relies on HPA to reduce misses (16KB Base page uses just 1 TLB)

F2FS large folios support

- Nanzhe Zhao's patchset

[f2fs-dev] [PATCH v2 00/14] f2fs: support & optimize large folios for writable files

Overall, order-2 and batched reads significantly improve performance, especially for larger I/O sizes. Write performance also improves substantially with background writeback

```
Nanzhe Zhao (14):
  f2fs: extend folio state for large folio write path
  f2fs: carry subpage offset and count in write IO
  f2fs: support regular file buffered writes on large folios
  f2fs: support atomic file large folios buffered write
  f2fs: support large folio writeback
  f2fs: prepare mmap write faults for large folios
  f2fs: make GC migration large-folio aware
  f2fs: optimize small block size large folio read
  f2fs: support partial uptodate large folio read
  f2fs: handle partial truncate of large folio dirty subpages
  f2fs: fix zeroing paths for large folios
  f2fs: handle block cloning within the same large folio
  f2fs: allow large folio support to writeable files
  f2fs: make compressed files compatible with large folio
```

```
Table 1: HOLE_READ (10 repeats)
------------------------------------------------------------
All bandwidth numbers are in MiB/s. Non-baseline entries show the
absolute value followed by the percentage delta relative to the
order=0 baseline in parentheses.

| bs   | order=0 | order=2 |
|------|---------|---------|
| 4k   | 469.6   | 521.9 (+11.1%) |
| 64k  | 668.1   | 852.4 (+27.6%) |
| 1M   | 653.0   | 867.2 (+32.8%) |

Table 2: DATA_READ (10 repeats)
---------------------------------------
| bs   | order=0 | batch=0 | batch=1,skip=0 | batch=1,skip=1 |
|------|---------|---------|----------------|----------------|
| 4k   | 441.6   | 456.5 (+3.4%) | 499.7 (+13.2%) | 544.4 (+23.3%) |
| 64k  | 632.8   | 697.0 (+10.1%) | 837.8 (+32.4%) | 990.9 (+56.6%) |
| 1M   | 601.5   | 733.0 (+21.9%) | 927.5 (+54.2%) | 963.4 (+60.2%) |

Table 3: WRITE (10 reps)
------------------------------------------------------------
O = overwrite (N = new write, Y = overwrite)
S = sync / fsync (Y = fsync enabled, N = no fsync)
W = writeback (Y = background writeback, N = no writeback)

| O,S,W | bs   | order=0 | order=2 |
|-------|------|---------|---------|
| N,N,Y | 4k   | 263.4   | 286.3 (+8.7%) |
| N,N,Y | 64k  | 683.5   | 1199.6 (+75.5%) |
| N,N,Y | 1M   | 767.4   | 1383.8 (+80.3%) |
```

order-0 alloc distribution with 16KB anon and file large folios

Order-0 callsite distribution (trace sampling, 1,945,684 samples ≈ 15.3% of totals)

Callsite group	Samples	Share
UFFD single-page fill (`mfill_atomic_pte_copy()`)	584,898	30.06%
Anonymous single-page path (`alloc_anon_folio()` / `folio_prealloc()`)	440,541	22.64%
COW / write-protect single-page (`do_wp_page()`)	353,580	18.17%
Trace buffer itself (`__rb_allocate_pages()`)	132,114	6.79%
TLB / page-table teardown (`__tlb_remove_folio_pages()` / `tlb_remove_table()`)	80,066	4.12%
Swapping-in single page (`do_swap_page()`)	71,112	3.65%
Mali GPU page pool (`kbase_mem_pool_alloc_pages()`)	67,065	3.45%
Page-table alloc (`pte_alloc_one_noprof()` / `__pmd_alloc()`)	62,176	3.20%
Other small items (`alloc_pages_bulk_noprof()` / `binder_alloc_new_buf()` / `dmabuf_page_pool_alloc()`)	60,210	3.09%
Process / task metadata (`dup_task_struct()` / `copy_process()`)	46,474	2.39%
File page / fs metadata (`__filemap_get_folio()` / `page_cache_ra_unbounded()`)	29,198	1.50%
Slab (`allocate_slab()`)	10,396	0.53%
zram / zsmalloc (`zs_malloc()`)	6,130	0.32%

Preliminary performance results

- 16K:order-2 mTHP always, defrag mode **madvise**(no kswapd/no direct reclaim for order-2) + F2FS large folios (limit max order to order-2 for file page caches and disable direct-reclaim for order-2)
- Baseline: Mainline

reboot the device, then cold-launch 18 apps; app cold launch time is the sum of the 18 apps' cold-start times

each app's cold-start time is launch-to-first-frame as measured by am start -W.

App cold launch (s)

	Mean	SD	Median	P90	Max
16K	**48.74**	1.12	48.81	49.75	50.80
Baseline	**52.16**	2.52	51.71	54.97	57.40
delta	**−6.6%**	−55%	−5.6%	−9.5%	−11.5%

kswapd on-CPU time (s)

	Mean	SD	Median	P90	Max
16K	**24.86**	2.00	24.19	27.74	28.14
Baseline	**27.47**	1.66	27.50	29.58	30.33
delta	**−9.5%**		−12.0%	−6.2%	−7.2%

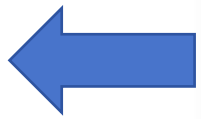
mTHP-friendly lightweight compaction

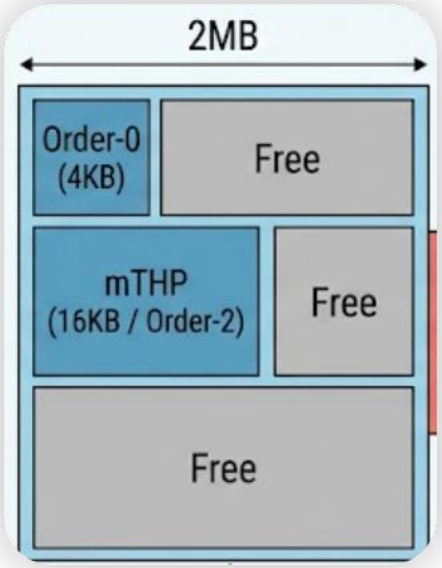
Bo Zhang's RFC v2:
[RFC PATCH v2 0/4] mm: compaction: mTHP-friendly memory compaction

- Compaction Mismatch:

2MB proactive compaction is expensive overkill for 16KB mTHP needs.

Even existing mTHP folios are forced to migrate alongside order-0 pages.





- Lightweight compaction:

Skips existing large folios (page cache and mTHP) to avoid unnecessary migration of already contiguous blocks.

change scudo’s page size to 16KB?

```
#define SCUDO_PAGE_SIZE 16_KB //with 4KB base page
```

- Benefits: Lower fragmentation
- Cost: Increased memory waste from:

✓Secondary Padding: mmap calls forced to 16KB alignment.

✓Deferred Freeing: Allocators cannot release memory to the OS until a full 16KB block accumulates.

- The cost may also apply to systems using 16KB base pages.

Potential optimization opportunity?

scudo using 16KB page size

#define SCUDO_PAGE_SIZE 16_KB //with 4KB base page

Conclusions

1. Aligning Scudo's page size to 16 KB makes Scudo VMAs 16 KB aligned, so 16 KB mTHP can be allocated for them.
2. It currently has a memory overhead under 16 KB alignment.

Measurements

12 native apps held in memory, no release

Metric (MB)	A (4 KB Scudo)	B (16 KB Scudo)	Δ (B−A)	Δ%
Scudo primary RSS	1979.8	1984.6	+4.8	+0.24%
Scudo secondary RSS	1600.5	1861.8	+261.3	+16.33%
Scudo RSS (total)	**3580.2**	**3846.4**	**+266.2**	**+7.44%**

The extra memory comes from Scudo secondary: round-up padding at 16 KB granularity instead of 4 KB.

After releasing Scudo memory (80% freed)

Metric (MB)	A (4 KB Scudo)	B (16 KB Scudo)	Δ (B−A)	Δ%
Scudo primary RSS	756.1	822.5	+66.4	+8.78%
Scudo secondary RSS	319.4	332.1	+12.7	+3.98%
Scudo RSS (total)	**1075.5**	**1154.6**	**+79.1**	**+7.35%**

The extra memory comes from Scudo primary fragmentation: a 16 KB page that still holds a live block cannot be released.

Discussions

- Disable direct reclaim for both page cache and mTHP higher-order allocations? Right now, this relies on Android hooks to modify the GFP flags.

```
void page_cache_ra_order(struct readahead_control *ractl,
                struct file_ra_state *ra, unsigned int new_order)
{
        ...
        if (!mapping_large_folio_support(mapping) || ra->size < 4)
                goto fallback;

        trace_android_vh_page_cache_ra_order_bypass(ractl, ra, new_order, &gfp,
                                                    &bypass);
        if (bypass)
                goto fallback;
```

- lightweight compaction: more use cases common to server and cloud workloads?
- Scudo: Can we avoid padding in the secondary allocator?
- ...